

Project 3: MongoDB

Asma Razick

G01310157

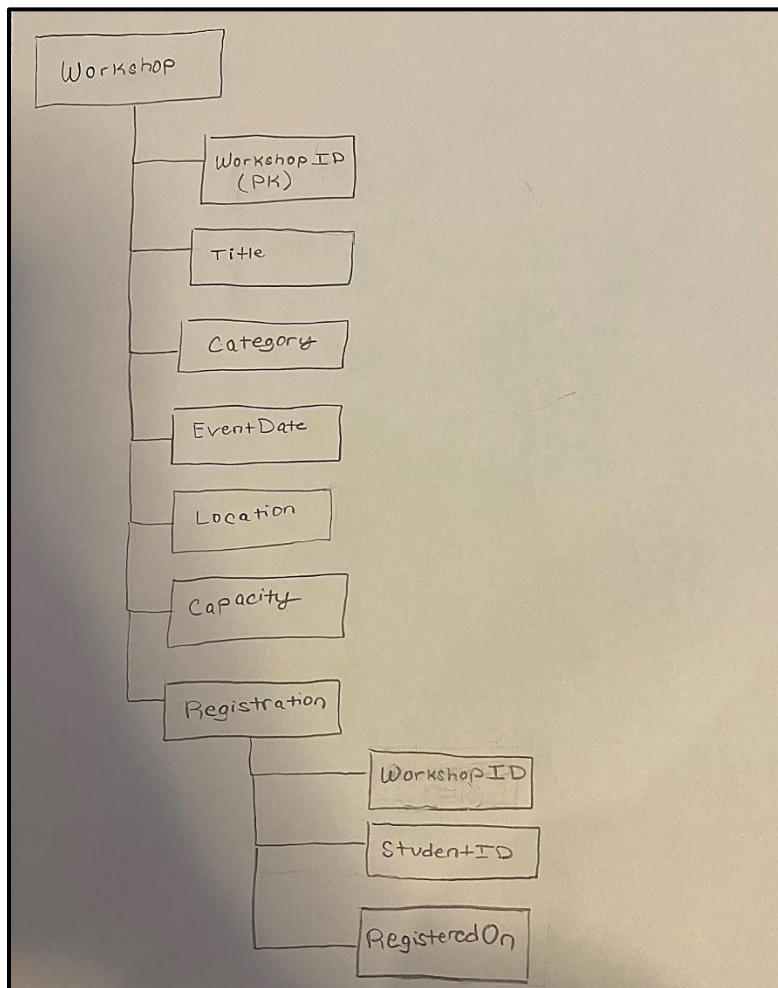
Professor Ping Deng

CS 450-001

8 December 2025

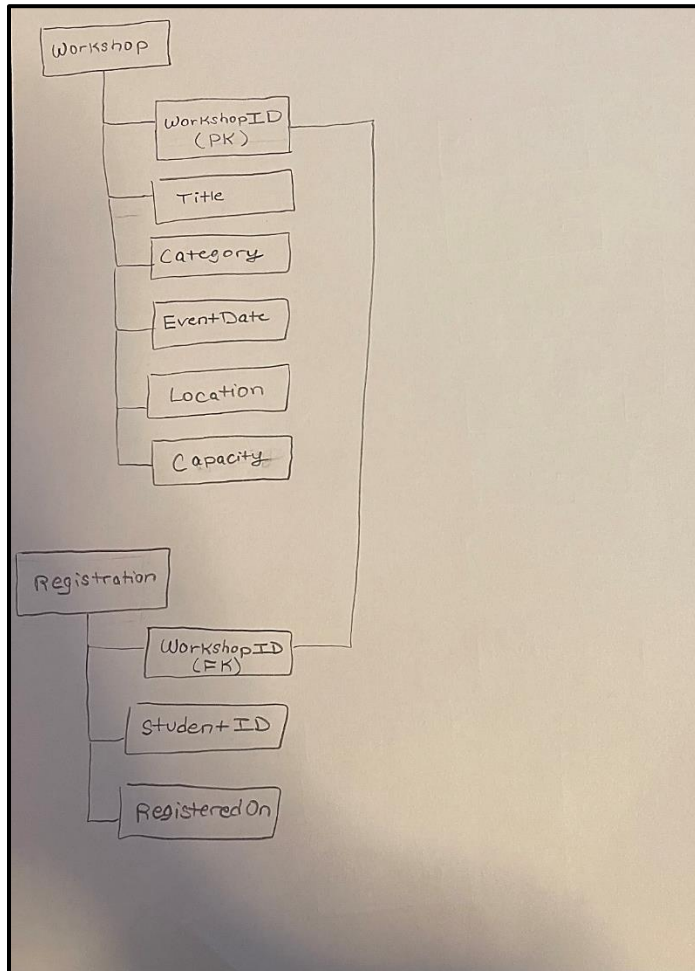
Part 1: Data Modeling

For this project there were two possible MongoDB schemas for the workshop registration data. Each of these schemas had their advantages and disadvantages. The first option was to embed registrations inside workshops. This schema uses one collection to store the workshops and registrations, meaning registrations can be accessed within the workshops collection. The primary benefits of this approach are that it minimizes redundancy and consolidates related data. This approach is also useful for simplifying queries, removing the need for nested queries and joins. However, this schema can soon become difficult to manage as more entries are added. As the collection size increases, the search time also increases. Below is a diagram depicting the embedding approach:



The second option was to link the registrations to the workshops. Rather than consolidating the data into one collection, this approach uses two separate collections. The workshops are stored in one collection, while the registrations are stored in another. The two collections are linked by the foreign key reference in registrations to the workshopID of the corresponding workshop. This approach is simpler to implement and balances the space taken

up, reducing search times. The disadvantage of this approach is that it can be more susceptible to redundancy. It also requires more complex queries and joins when querying the data. Below is a diagram depicting the linking approach:



For this project I decided to use the linking approach. This decision was based on examining the pros and cons of both approaches. Although querying would be more complex, the simplicity of migrating and storing the data led to my decision to use the linking approach.

Part 2: Data Migration and Document Counting

Now that I had decided on a schema, I needed to migrate the database from SQL to MongoDB. First, I logged into Zeus and ran the command `'sqlplus'` to log into my SQL account. Then, I ran the command `'SPOOL migration.js'`. After that, I ran the following script so that it would produce a .js file with all the commands to run in MongoDB. The script is shown in the following image:

```
--Name: Asma Razick
--GNumber: G01310157
--Section: CS 450-001

SET LINESIZE 1000;
SET PAGESIZE 0;
SET TRIMSPOOL ON;
SET FEEDBACK OFF;
SET ECHO OFF;
SET DEFINE OFF;
--Schema: Linking/Referencing

-- WORKSHOPS MIGRATION
SELECT 'db.workshops.insertOne({' ||
'WorkshopID: ' || WorkshopID || ', ' ||
'Title: ' || Title || ', ' ||
'Category: ' || Category || ', ' ||
'EventDate: ' || TO_CHAR(EventDate, 'YYYY-MM-DD') || ', ' ||
'Location: ' || Location || ', ' ||
'Capacity: ' || Capacity ||
'})';
FROM Workshops;

-- REGISTRATIONS MIGRATION
SELECT 'db.registrations.insertOne({' ||
'WorkshopID: ' || WorkshopID || ', ' ||
'StudentID: ' || StudentID || ', ' ||
'RegisteredOn: ' || TO_CHAR(RegisteredOn, 'YYYY-MM-DD') || ', ' ||
'})';
FROM Registrations;
```

After running the script, I switched over to command prompt and ran the migration.js file in order to populate the database in Mongo. Below are screenshots of the execution of the migration.js file after running it in Mongo:

```
C:\Users\azra>mongosh "mongodb://localhost:27017/workshops_registrations" < "C:\Users\azra\Documents\College\FALL 2025\CS 450\Project 3\migration.js"
Current Mongosh Log ID: 692f3805489ea42c441a2620
Connecting to:      mongodb://localhost:27017/workshops_registrations?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.10
Using MongoDB:      8.2.2
Using Mongosh:       2.5.10

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2025-11-30T16:24:25.836-05:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

workshops_registrations>
workshops_registrations> db.workshops.insertOne({ WorkshopID: 101, Title: "Intro to Data Visualization", Category: "Tech", EventDate: "2025-11-05", Location: "Innovation Hall 204", Capacity: 40 });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a2621') }
workshops_registrations> db.workshops.insertOne({ WorkshopID: 102, Title: "Resume & Cover Letter Clinic", Category: "Career", EventDate: "2025-11-06", Location: "Johnson Center 3rd Fl", Capacity: 50 });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a2622') }
workshops_registrations> db.workshops.insertOne({ WorkshopID: 103, Title: "Mindfulness for Midterms", Category: "Wellness", EventDate: "2025-11-07", Location: "SUB I Room 1201", Capacity: 30 });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a2623') }
workshops_registrations> db.workshops.insertOne({ WorkshopID: 104, Title: "Git & GitHub Basics", Category: "Tech", EventDate: "2025-11-08", Location: "Engineering 110", Capacity: 35 });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a2624') }
workshops_registrations> db.workshops.insertOne({ WorkshopID: 105, Title: "Networking for Internships", Category: "Career", EventDate: "2025-11-10", Location: "Career Center Hall A", Capacity: 60 });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a2625') }
workshops_registrations> db.workshops.insertOne({ WorkshopID: 106, Title: "Public Speaking Fundamentals", Category: "Career", EventDate: "2025-11-12", Location: "Johnson Center Cinema", Capacity: 90 });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a2626') }
workshops_registrations> db.workshops.insertOne({ WorkshopID: 107, Title: "Stretch & Study: Desk Mobility", Category: "Wellness", EventDate: "2025-11-13", Location: "Recreation Center Studio 8", Capacity: 25 });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a2627') }
workshops_registrations> db.workshops.insertOne({ WorkshopID: 108, Title: "Time Management for Finals", Category: "Wellness", EventDate: "2025-11-15", Location: "SUB I Room 1101", Capacity: 40 });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a2628') }
workshops_registrations> db.registrations.insertOne({ WorkshopID: 101, StudentID: "S012300001", RegisteredOn: "2025-10-28" });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a2629') }
workshops_registrations> db.registrations.insertOne({ WorkshopID: 101, StudentID: "S012300002", RegisteredOn: "2025-10-28" });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a262a') }
workshops_registrations> db.registrations.insertOne({ WorkshopID: 101, StudentID: "S012300003", RegisteredOn: "2025-10-29" });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a262b') }
workshops_registrations> db.registrations.insertOne({ WorkshopID: 102, StudentID: "S012300001", RegisteredOn: "2025-10-29" });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a262c') }
workshops_registrations> db.registrations.insertOne({ WorkshopID: 102, StudentID: "S012300004", RegisteredOn: "2025-10-30" });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a262d') }
workshops_registrations> db.registrations.insertOne({ WorkshopID: 102, StudentID: "S012300005", RegisteredOn: "2025-10-30" });
{ acknowledged: true,
  insertedId: ObjectId('692f3805489ea42c441a262e') }
```

```

workshops_registrations> db.registrations.insertOne({ WorkshopID: 103, StudentID: "001130000", RegisteredOn: "2025-10-30" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a262f' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 103, StudentID: "001130000", RegisteredOn: "2025-10-31" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a2630' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 104, StudentID: "001130000", RegisteredOn: "2025-10-31" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a2631' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 104, StudentID: "001130000", RegisteredOn: "2025-10-31" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a2632' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 104, StudentID: "001130000", RegisteredOn: "2025-11-01" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a2633' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 105, StudentID: "001130001", RegisteredOn: "2025-11-01" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a2634' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 105, StudentID: "001130009", RegisteredOn: "2025-11-01" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a2635' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 105, StudentID: "001130010", RegisteredOn: "2025-11-02" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a2636' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 106, StudentID: "001130004", RegisteredOn: "2025-11-02" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a2637' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 106, StudentID: "001130011", RegisteredOn: "2025-11-02" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a2638' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 106, StudentID: "001130012", RegisteredOn: "2025-11-03" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a2639' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 107, StudentID: "001130002", RegisteredOn: "2025-11-03" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a263a' )
}
workshops_registrations> db.registrations.insertOne({ WorkshopID: 107, StudentID: "001130013", RegisteredOn: "2025-11-03" });
{
  acknowledged: true,
  insertedId: ObjectId( '692f3805409ea42c441a263b' )
}
workshops_registrations>
workshops_registrations>
C:\Users\lauraz>

```

Finally, in order to ensure that all the data was migrated, I ran the following commands:

```

workshops_registrations> db.workshops.countDocuments()
8
workshops_registrations> db.registrations.countDocuments()
19

```

Now that I had data in MongoDB, I could officially start running queries and move forward with Part 3.

Part 3: Querying the MongoDB Database

1. Find the titles and event dates of all workshops in the “Tech” category.

```
workshops_registrations> db.workshops.find({Category: 'Tech'}, {Title: 1, EventDate: 1, _id: 0})
[
  { Title: 'Intro to Data Visualization', EventDate: '2025-11-05' },
  { Title: 'Git & GitHub Basics', EventDate: '2025-11-08' }
]
```

2. Find the workshopIDs, titles, and capacities of all “Career” workshops whose capacity is at least 60.

```
workshops_registrations> db.workshops.find({Category: 'Career', Capacity: 60}, {WorkshopID: 1, Title: 1, Capacity: 1, _id: 0})
[
  {
    WorkshopID: 105,
    Title: 'Networking for Internships',
    Capacity: 60
  }
]
```

3. Find the titles of the workshops that are held in either Innovation Hall 204 or Engineering 110.

```
workshops_registrations> db.workshops.find({Location: {$in: ['Innovation Hall 204', 'Engineering 110']}},{Title: 1, _id: 0})
[
  { Title: 'Intro to Data Visualization' },
  { Title: 'Git & GitHub Basics' }
]
```

4. Find the workshopIDs of the “Wellness” workshops whose eventDate is on or after 2025-11-13.

```
workshops_registrations> db.workshops.find({Category: 'Wellness', EventDate: {$gte: "2025-11-13"}}, {WorkshopID: 1, _id: 0})
[ { WorkshopID: 107 }, { WorkshopID: 108 } ]
```

5. Find the titles of the workshops that currently have no registrations.

```
workshops_registrations> db.workshops.aggregate([{$lookup: {from: "registrations", localField: "WorkshopID", foreignField: "WorkshopID", as: "registration"}},{$match: {registration: {$size: 0}}},{project: {Title: 1, _id: 0}}])
[ { Title: 'Time Management for Finals' } ]
workshops_registrations>
```

6. Find the workshopIDs of the workshops that have at least 3 students registered.

```
workshops_registrations> db.workshops.aggregate([{$lookup: {from: "registrations", localField: "WorkshopID", foreignField: "WorkshopID", as: "registration"}},{$match: {registration: {$size: 3}}},{project: {WorkshopID: 1, _id: 0}}])
[
  { WorkshopID: 101 },
  { WorkshopID: 102 },
  { WorkshopID: 104 },
  { WorkshopID: 105 },
  { WorkshopID: 106 }
]
```

7. Find the titles of the workshops where the first registered student is NOT “G01230001”

```
workshops_registrations> db.workshops.aggregate([{$lookup: {from: 'registrations', localField: 'WorkshopID', foreignField: 'WorkshopID', as: 'registration'}},{$match: {'registration.0': { $exists: true },'registration.0.StudentID': { $ne: 'G01230001' }}}),{$project: { Title: 1, _id: 0 }}])
[
  { Title: 'Mindfulness for Midterms' },
  { Title: 'Git & Github Basics' },
  { Title: 'Public Speaking Fundamentals' },
  { Title: 'Stretch & Study: Desk Mobility' }
]
```

8. Find the studentIDs of the students who are registered for at least two different workshops.

```
workshops_registrations> db.registrations.aggregate([{$group: {_id: "$StudentID", workshopCount: { $sum: 1 }}}),{$match: {workshopCount: { $gte: 2 }}},{$project: { _id: 1 }}])
[
  { _id: 'G01230004' },
  { _id: 'G01230002' },
  { _id: 'G01230001' },
  { _id: 'G01230003' }
]
```